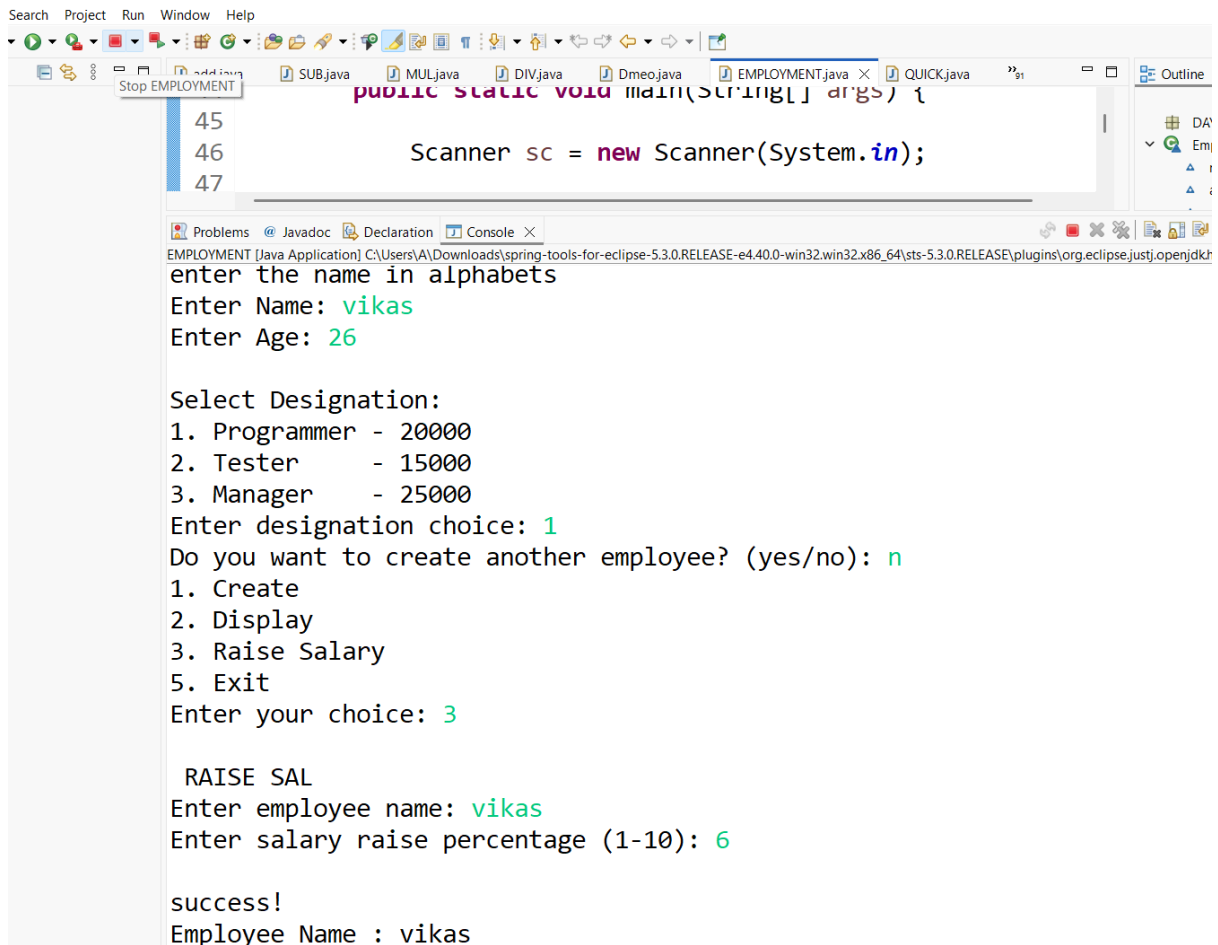




```
public static void main(String[] args) {
    45
    46         Scanner sc = new Scanner(System.in);
    47
    enter the name in alphabets
    Enter Name: vikas
    Enter Age: 26

    Select Designation:
    1. Programmer - 20000
    2. Tester      - 15000
    3. Manager     - 25000
    Enter designation choice: 1
    Do you want to create another employee? (yes/no): n
    1. Create
    2. Display
    3. Raise Salary
    5. Exit
    Enter your choice: 3

    RAISE SAL
    Enter employee name: vikas
    Enter salary raise percentage (1-10): 6

    success!
    Employee Name : vikas
```

EXPLANATION

This Java program is an **Employee Management System** that uses a HashMap to store employee details. It provides **four options: Create, Display, Raise Salary, and Exit**. The program uses switch, loops, Scanner, input validation, and an Employee class to manage the employees.

Employee Class

The Employee class stores four details: **name, age, designation, and salary**. The constructor is used to initialize these values when a new employee is created.

Case 1 – Create

Case 1 calls the create() method. It asks the user to enter the employee's **name, age, and designation**. The name is checked so that it does not contain more than two spaces and the same employee name cannot be entered twice. The age must be between **18 and 60**. The designation must be **Programmer, Manager, or Tester**. Based on the designation, the starting salary is automatically assigned: Programmer – ₹20,000, Manager – ₹25,000, and Tester – ₹15,000. Finally, the employee is stored in the HashMap. The user can also choose to add another employee.

Case 2 – Display

Case 2 calls the `display()` method. It first checks whether there are any employees in the `HashMap`. If there are no employees, it displays a message saying that there are no employees to display. Otherwise, it uses a **for-each loop** to go through all employees and displays their **name, age, designation, and salary**.

Case 3 – Raise Salary

Case 3 calls the `raiseSalary()` method. It first checks whether any employees exist. Then it asks the user to enter the **name of the employee** whose salary should be increased. The program searches for that employee in the `HashMap`. If the name is not found, it asks the user to enter the name again. After finding the employee, it asks for a salary increase between **1% and 10%**. The new salary is calculated and updated. For example, if the salary is ₹20,000 and the increase is 10%, the new salary becomes ₹22,000.

Case 4 – Exit

Case 4 exits the program. It prints **"EXIT"**, closes the `Scanner`, and uses `return` to stop the `main()` method.

Input Validation

The program also validates numeric inputs using `hasNextInt()` and `hasNextDouble()`. This prevents the program from crashing when the user enters invalid data such as letters instead of numbers. The `while` and `do-while` loops repeatedly ask for input until the user enters valid information.